Chapter 7

Run Time Enviorments

Runtime Environments

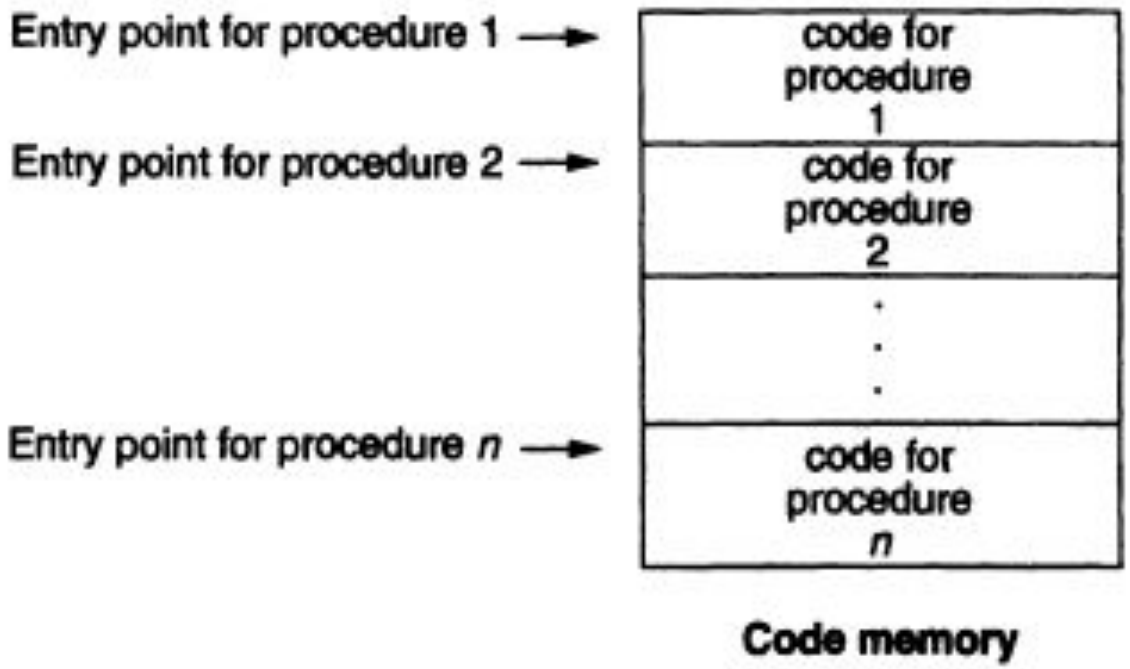
- Manages memory, registers, and resources during program execution.
- Handles memory allocation, procedure execution, and variable scoping.

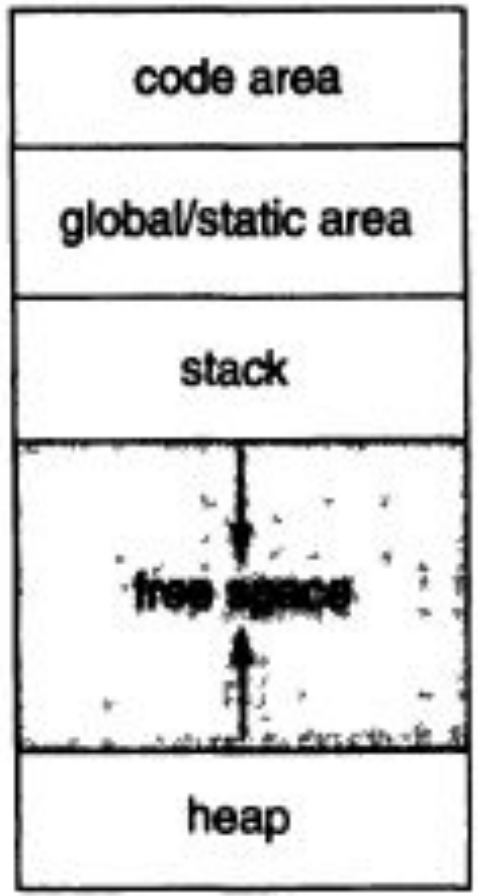
- **Three Runtime Environment Types**:

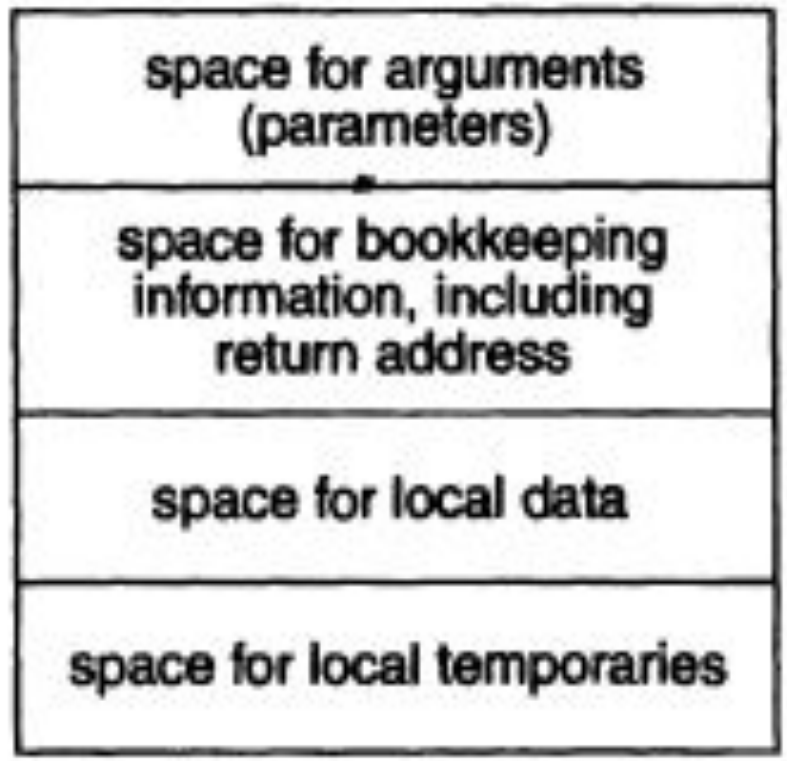
1. **Fully Static**:
   - Pre-allocated memory (e.g., FORTRAN77).
   - No recursion or dynamic allocation.
2. **Stack-Based**:
   - Supports recursion via activation records (e.g., C, Pascal).
3. **Fully Dynamic**:
   - Runtime allocation with garbage collection (e.g., LISP, Java).

Runtime enviorment

- **Design Importance**:
  - Manages variable scoping and memory allocation.
  - Implements parameter-passing mechanisms (call-by-value, call-by-reference).
  - Optimizes program execution on the target machine.
- **Compiler's Role**:
  - Generates code to manage the runtime environment.
  - Ensures compatibility with OS and hardware.
- *Sets the foundation for exploring memory organization, calling conventions, and dynamic allocation.*


Entry point for procedure 1
Entry point for procedure 2
Entry point for procedure n
code for procedure 1
code for procedure 2
code for procedure n
Code memory

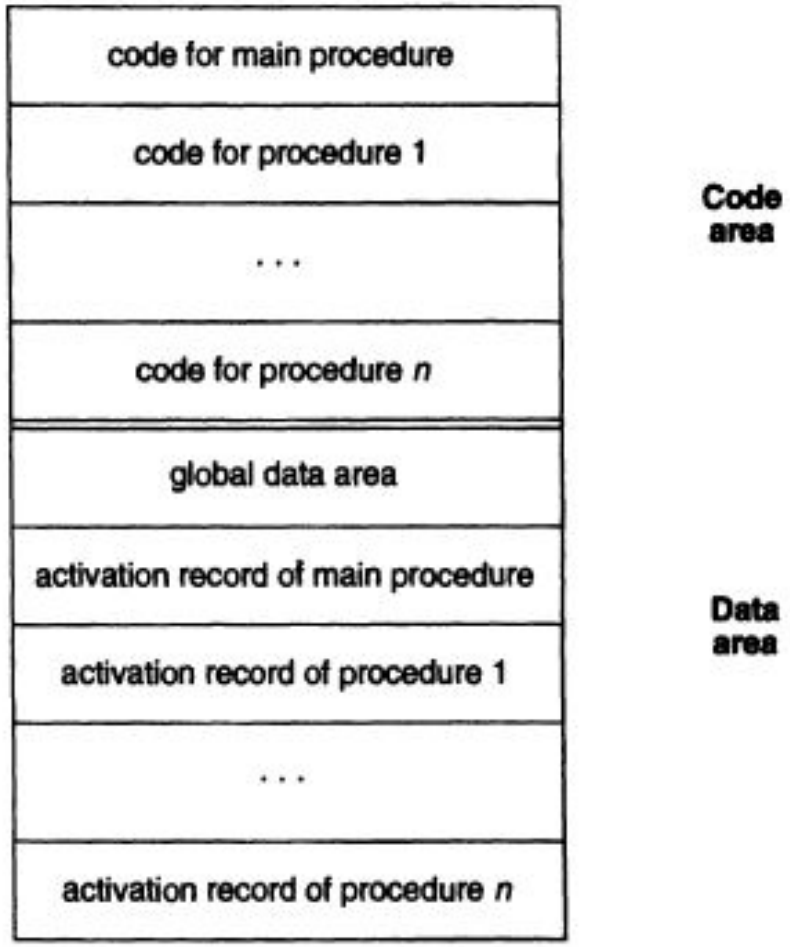

code area
global/static area
stack
free space
heap


space for arguments (parameters)
space for bookkeeping information, including return address
space for local data
space for local temporaries

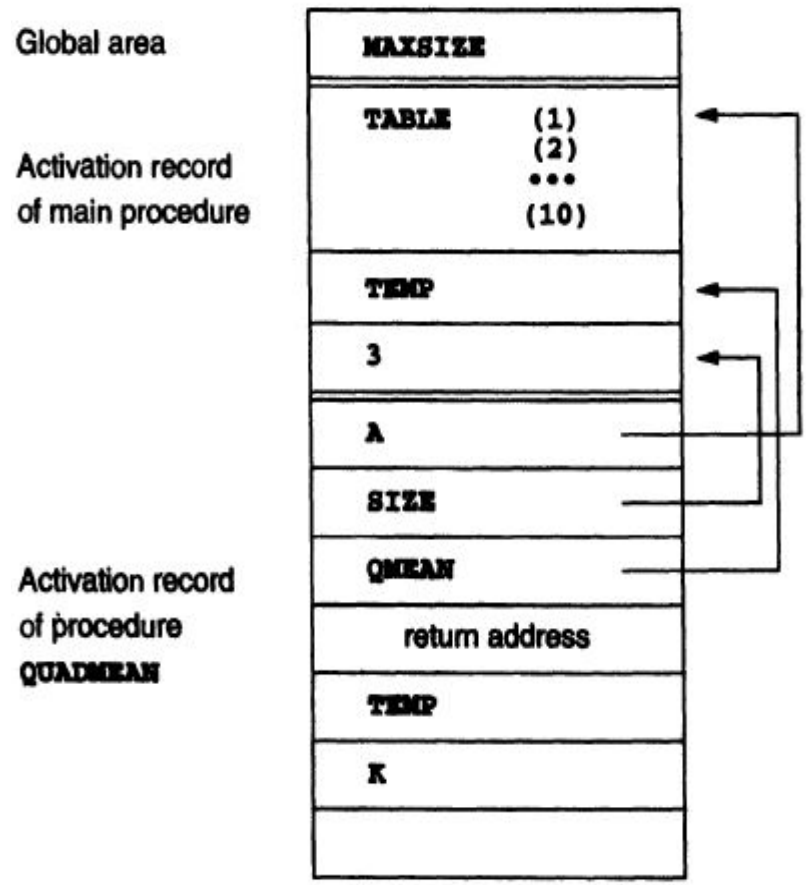
Memory Layout

Memory Area	Contents
Code	Executable instructions
Static Data	Global variables, constants
Stack	Activation records, local variables
Heap	Dynamically allocated objects
Free Space	Available for future allocation

Fully Static Runtime Environments



```
      PROGRAM TEST
      COMMON MAXSIZE
      INTEGER MAXSIZE
      REAL TABLE(10),TEMP
      MAXSIZE = 10
      READ *, TABLE(1),TABLE(2),TABLE(3)
      CALL QUADMEAN(TABLE,3,TEMP)
      PRINT *, TEMP
      END

      SUBROUTINE QUADMEAN(A,SIZE,QMEAN)
      COMMON MAXSIZE
      INTEGER MAXSIZE,SIZE
      REAL A(SIZE),QMEAN, TEMP
      INTEGER K
      TEMP = 0.0
      IF ((SIZE.GT.MAXSIZE).OR.(SIZE.LT.1)) GOTO 99
      DO 10 K = 1,SIZE
         TEMP = TEMP + A(K)*A(K)
10    CONTINUE
99    QMEAN = SQRT(TEMP/SIZE)
      RETURN
      END
```

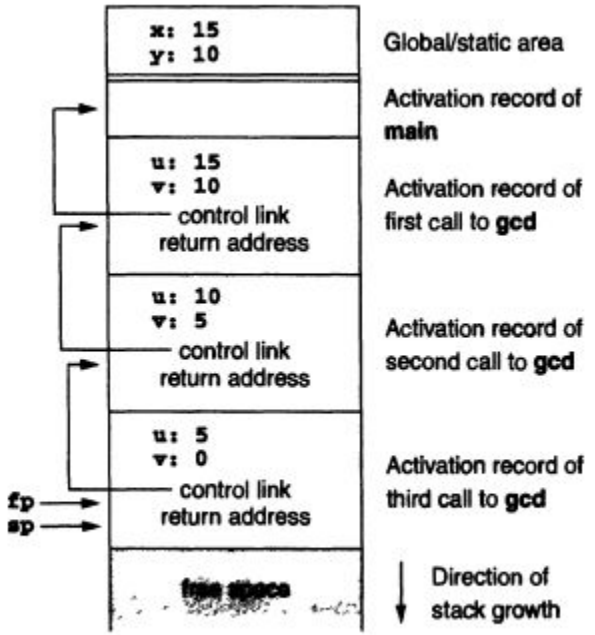


Stack based Run time Environments

Figure 7.3
C code for Example 7.2

```
#include <stdio.h>

int x,y;

int gcd( int u, int v)
{ if (v == 0) return u;
  else return gcd(v,u % v);
}

main()
{ scanf("%d%d",&x,&y);
  printf("%d\n",gcd(x,y));
  return 0;
}
```

Figure 7.4
Stack-based environment for Example 7.2



Example 7.3

Figure 7.5
C program of Example 7.3

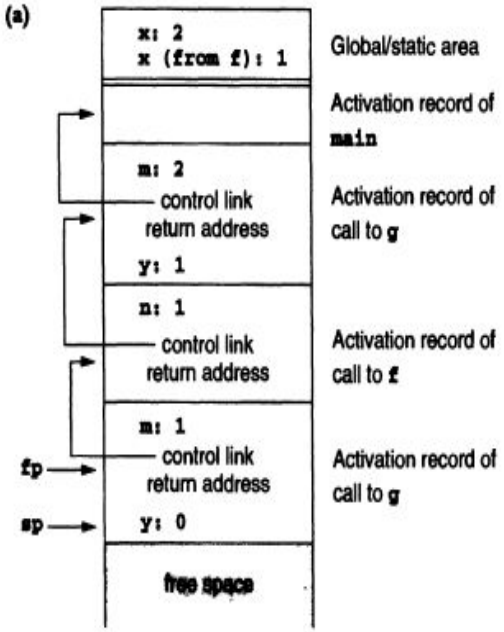
```
int x = 2;

void g(int); /* prototype */

void f(int n)
{ static int x = 1;
  g(n);
  x--;
}

void g(int m)
{ int y = m-1;
  if (y > 0)
  { f(y);
    x--;
    g(y);
  }
}


main()
{ g(x);
  return 0;
}
```

Example 7.3

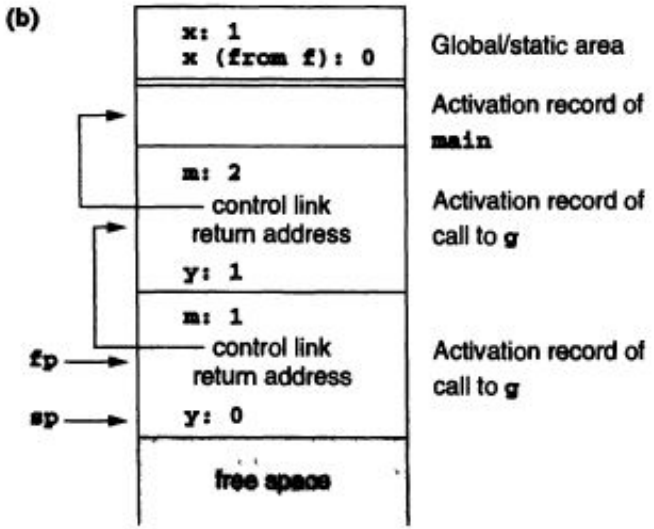
Figure 7.5
C program of Example 7.3

```
int x = 2;

void g(int); /* protot

void f(int n)
{ static int x = 1;
  g(n);
  x--;
}

void g(int m)
{ int y = m-1;
  if (y > 0)
  { f(y);
    x--;
    g(y);
  }
}

main()
{ g(x);
  return 0;
}
```

Figure 7.6
(a) Runtime environment of the program of Figure 7.5 during the second call to g



(b) Runtime environment of the program of Figure 7.5 during the third call to g



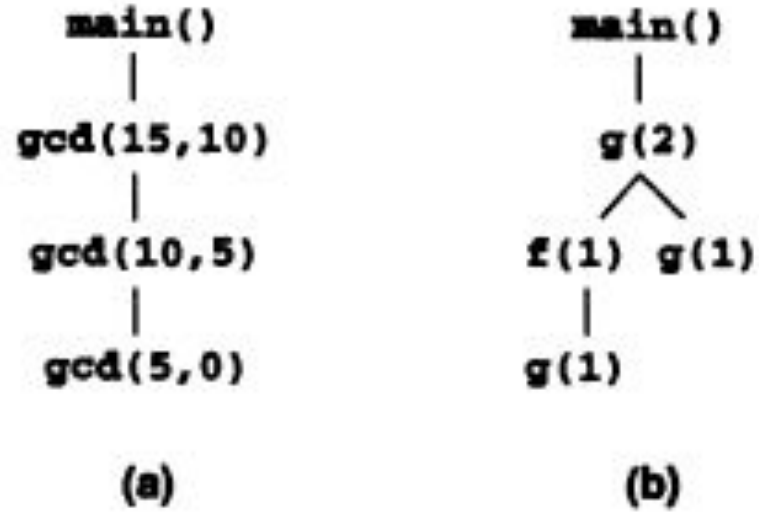
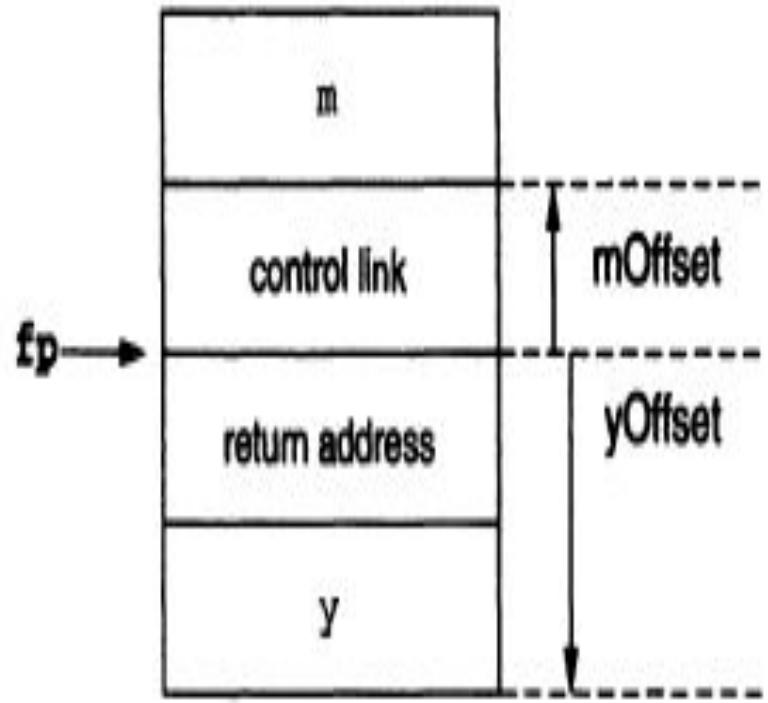


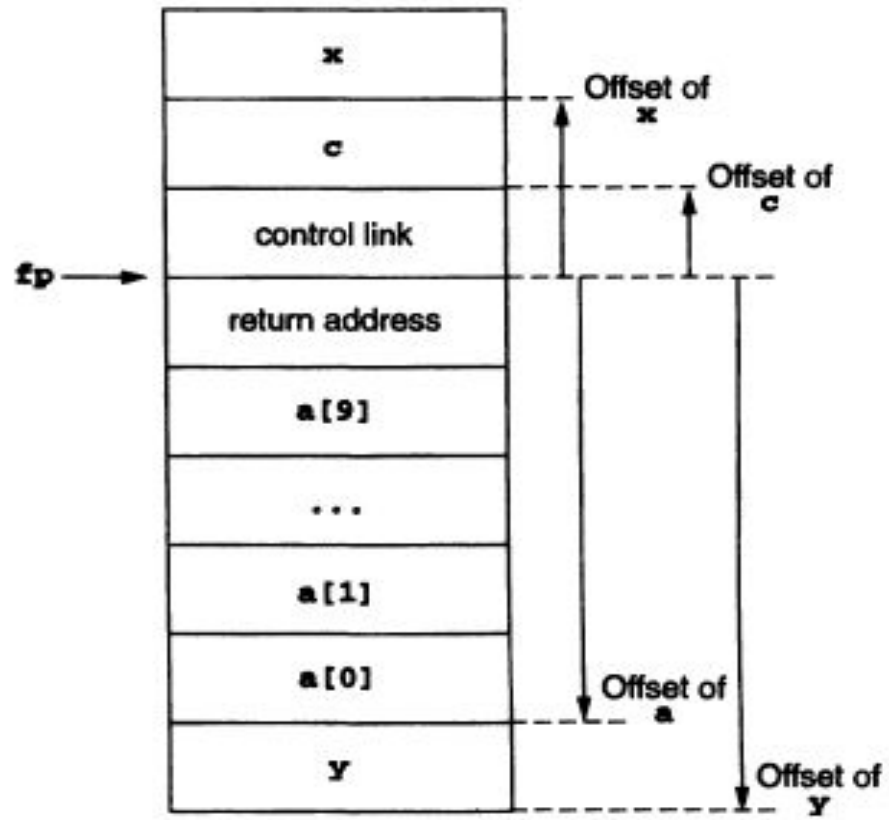
Figure 7.7
Activation trees for the programs of Figures 7.3 and 7.5



Example 7.4 Consider the C procedure

```
void f(int x, char c)
{ int a[10];
  double y;
  ...
}
```

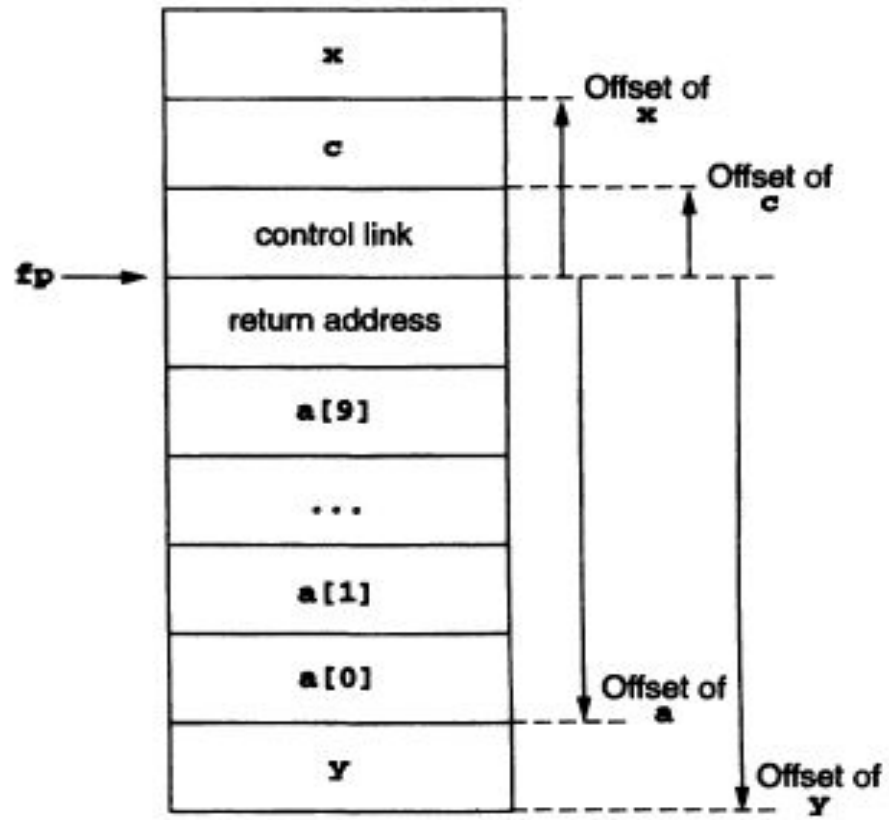
The activation record for a call to `f` would appear as



Example 7.4 Consider the C procedure

```
void f(int x, char c)
{ int a[10];
  double y;
  ...
}
```

The activation record for a call to `f` would appear as



Name	Offset
x	+5
c	+4
a	−24
y	−32

The Calling Sequence The calling sequence now comprises approximately the following steps.[8] When a procedure is called,

1. Compute the arguments and store them in their correct positions in the new activation record of the procedure (pushing them in order onto the runtime stack will achieve this).
2. Store (push) the fp as the control link in the new activation record.
3. Change the fp so that it points to the beginning of the new activation record (if there is an sp, copying the sp into the fp at this point will achieve this).
4. Store the return address in the new activation record (if necessary).
5. Perform a jump to the code of the procedure to be called.

When a procedure exits,

1. Copy the fp to the sp.
2. Load the control link into the fp.
3. Perform a jump to the return address.
4. Change the sp to pop the arguments.

Variable Length Data

object. Two examples that occur in languages that support stack-based environments are (1) the number of arguments in a call may vary from call to call, and (2) the size of an array parameter or a local array variable may vary from call to call.

```
printf("%d%s%c",n,prompt,ch);
```

has four arguments (including the format string `"%d%s%c"`), while

```
printf("Hello, world\n");
```

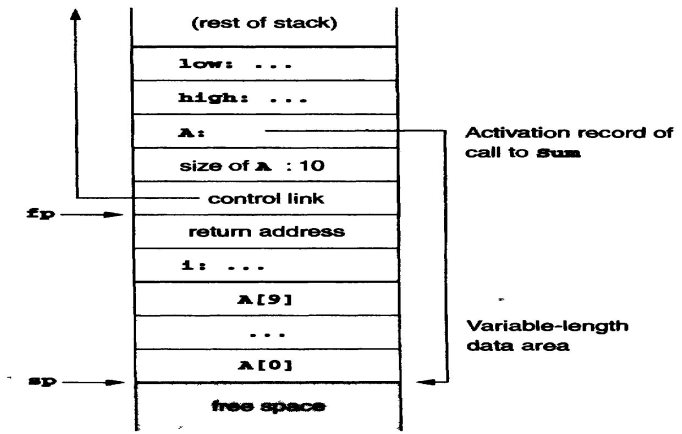
Parameter and local Variable variable length

An example of situation 2 is the **unconstrained array** of Ada:

```
type Int_Vector is
       array(INTEGER range <>) of INTEGER;

procedure Sum (low,high: INTEGER;
                      A: Int_Vector) return INTEGER
is
   temp: Int_Array (low..high);
begin
   ...
end Sum;
```

(Note the local variable **temp** which also has unpredictable size.) A typical method for dealing with this situation is to use an extra level of indirection for the variable-length data, storing a pointer to the actual data in a location that can be predicted at compile time, while performing the actual allocation at the top of the runtime stack in a way that can be managed by the sp during execution.

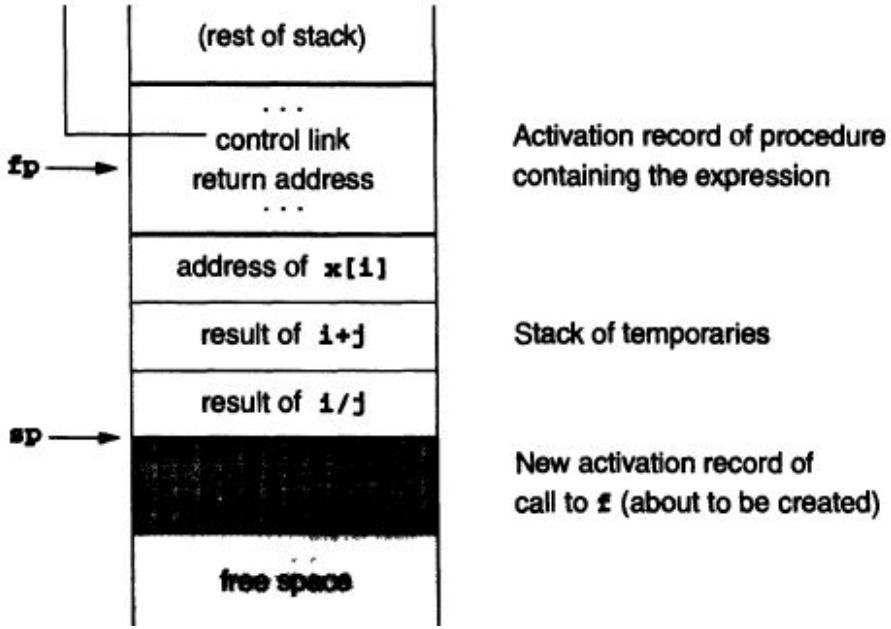
Given the Ada **Sum** procedure as defined above, and assuming the same organization for the environment as before,[9] we could implement an activation record for **Sum** as follows (this picture shows, for concreteness, a call to **Sum** with an array of size 10):



Local Temporaries

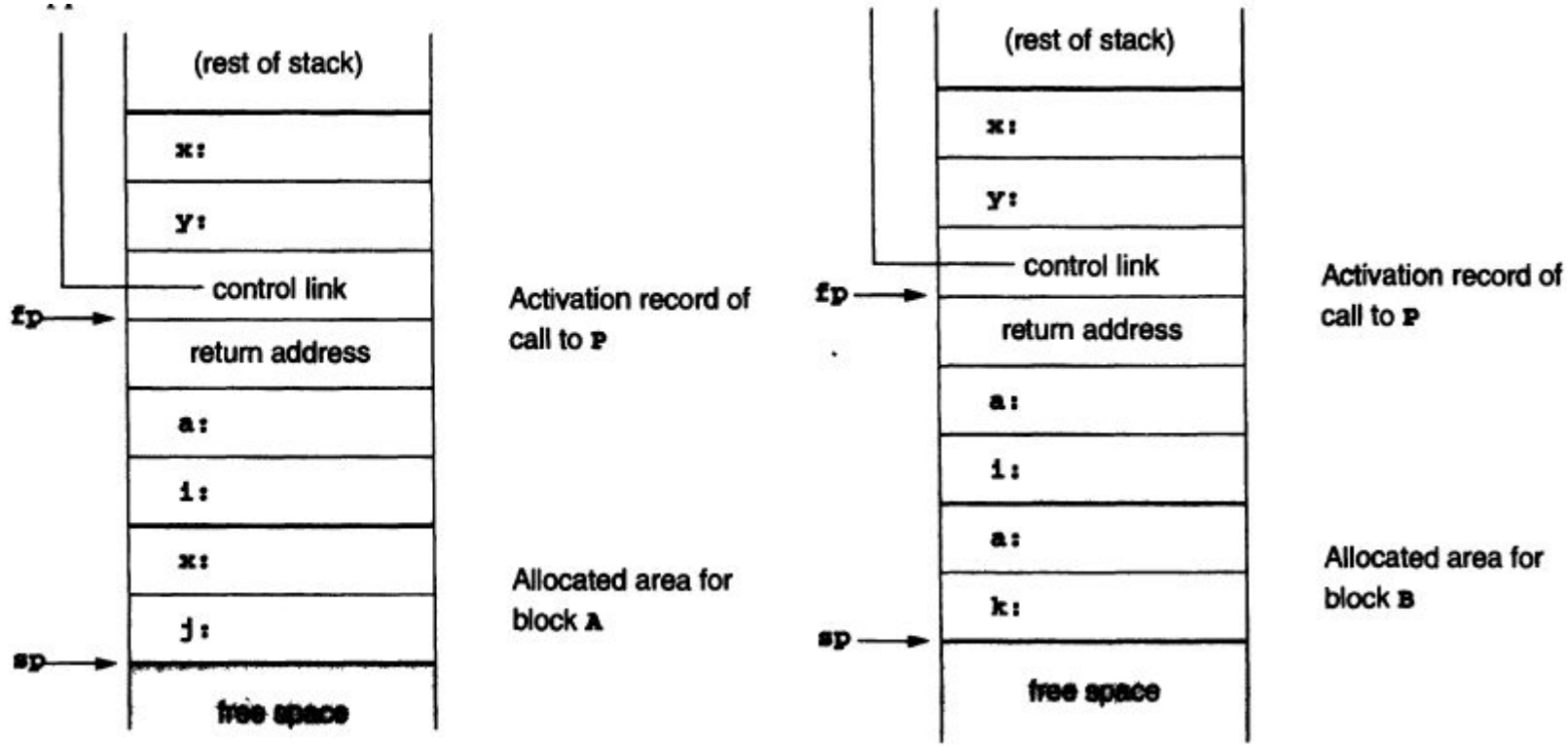
```
x[i] = (i + j)*(i/k + f(j))
```

In a left-to-right evaluation of this expression, three partial results need to be saved across the call to `f`: the address of `x[i]` (for the pending assignment), the sum `i+j` (pending the multiplication), and the quotient `i/k` (pending the sum with the result of the call `f(j)`). These partial results could be computed into registers and saved and restored according to some register management mechanism, or they could be stored as temporaries on the runtime stack prior to the call to `f`. In this latter case, the runtime stack might appear as follows at the point just before the call to `f`:



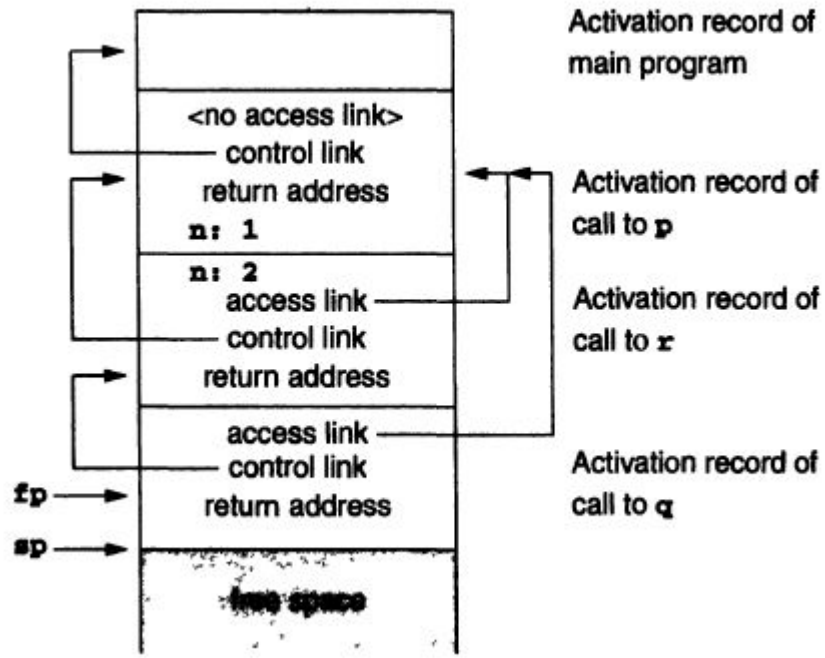
Nested Declaration

```
void p( int x, double y)
{ char a;
  int i;
  ...
A:{ double x;
    int j;
    ...
  }
  ...
B:{ char * a;
    int k;
    ...
  }
  ...
}
```

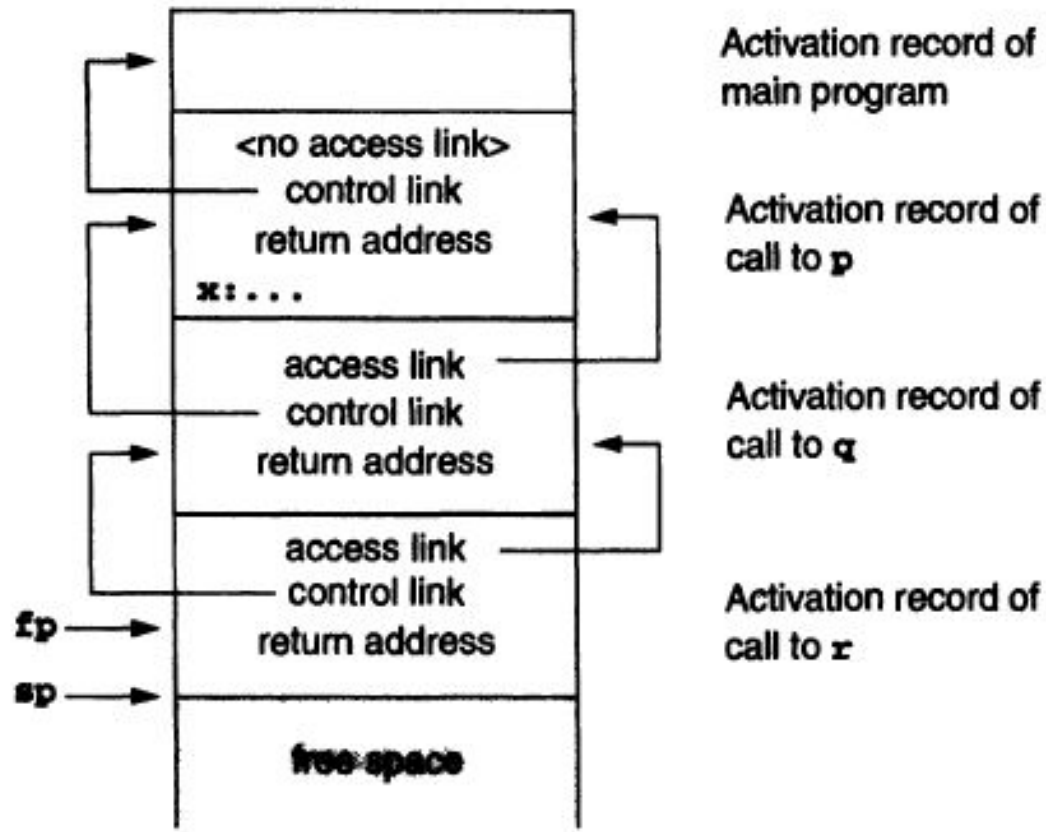


Stack Based Environments with local procedures

```
program nonLocalRef;

procedure p;
var n: integer;

      procedure q;
      begin
        (* a reference to n is now
          non-local non-global *)
      end; (* q *)

      procedure r(n: integer);
      begin
        q;
      end; (* r *)

begin (* p *)
  n := 1;
  r(2);
end; (* p *)

begin (* main *)
  p;
end.
```



```
program chain;

procedure p;
var x: integer;

   procedure q;
      procedure r;
      begin
        x := 2;
        ...
        if ... then p;
      end; (* r *)
   begin
     r;
   end; (* q *)

begin
  q;
end; (* p *)

begin (* main *)
  p;
end.
```



Dynamic Memory

- Memory allocated **at runtime** (not compile time).
  - Data structures with variable size (e.g., linked lists, trees).
  - Objects in OOP languages (e.g., Java, C++).
- **Memory Segments**:
  - **Heap**: Primary area for dynamic allocation.
  - **Stack**: Limited to function-local, fixed-size data.

Heap management

- Manages **dynamic memory allocation** (e.g., malloc, new).
- Handles variable-sized data structures (linked lists, trees).

- **Circular Linked List**:
  - Tracks **free blocks** and **allocated blocks** using a linked list.
  - **Header structure**: Stores metadata (block size, next pointer).
- **Allocation Process**:
  - Search free list for a block of sufficient size.
  - Split block if extra space remains.
- **Free Process**:
  - Mark block as free.
  - **Coalesce** adjacent free blocks to reduce fragmentation.

Garbage Collection

- **Calls when malloc fails**
- **Mark-and-Sweep**:
  - **Mark**: follow all pointers recursively and marks each block of storage reached.
  - **Sweep**: Free unmarked (unreachable) blocks.
- **Stop-and-Copy**:
  - Divide heap into two spaces; copy live objects to new space.
  - **Advantage**: Automatic compaction.
- **Generational GC**:
  - Prioritizes collection of short-lived objects (e.g., Java's Young/Old generations).

Parameter Passing Mechanism

Call-by-Value

- **Mechanism**:
  - Copies the **value** of the actual parameter to the formal parameter.
  - Changes to the formal parameter **do not affect** the actual parameter.
- **Implementation**:
  - Simplest to implement; no aliasing issues.
  - Used in C, Pascal, and Ada (default).

Call-by-Reference

- **Mechanism**:
  - Passes the **location** of the actual parameter.
  - Formal parameter becomes an **alias** for the actual parameter.
- **Implementation**:
  - Requires compiler to compute and pass addresses.
  - Used in Fortran77, Pascal (var keyword), and C++ (&).

Call-by-Value-Result (Copy-In/Copy-Out)

- **Mechanism**:
  - Copies the value **into** the formal parameter at call time.
  - Copies the result **back** to the actual parameter at return.
- **Implementation**:
  - Resembles pass-by-reference but avoids aliasing.
  - Used in Ada (in out parameters).

Call-by-Name (Delayed Evaluation)

- **Mechanism**:
  - Replaces every use of the formal parameter with the **unevaluated expression** of the actual parameter.
  - Evaluated **each time** it is referenced.

if a call such as `p(a[i])` is made, the effect is of evaluating `++(a[i])`. Thus, if `i` were to change before the use of `x` inside `p`, the result would be different from either pass by reference or pass by value-result. For instance, in the code (in C syntax),

```
int i;
int a[10];

void p(int x)
{ ++i;
  ++x;
}
```

```
main()
( i = 1;
   a[1] = 1;
   a[2] = 2;
   p(a[i]);
   return 0;
}
```

the result of the call to **p** is that **a[2]** is set to 3 and **a[1]** is left unchanged.